ИТМО
ВЫСШАЯ ШКОЛА
ЦИФРОВОЙ КУЛЬТУРЫ

Хранение данных и реляционные базы:

таблицы, ключи, SQL-запросы и проверка с ИИ

Данные как основа ИИ · Тема 08

План занятия

1. **Почему одной таблицы мало.** Повторы и аномалии ведомости.
2. **Таблицы, ключи и связи.** PK, FK, 1 : M и M : M, схема учебной базы.
3. **СУБД и SQL.** Как задать вопрос базе.
4. **Выборка.** SELECT, WHERE, ORDER BY, LIMIT и NULL.
5. **Агрегаты.** COUNT, AVG, GROUP BY, HAVING и порядок выполнения.
6. **Соединения.** JOIN, LEFT JOIN и размножение строк.
7. **SQL вместе с ИИ.** Генерация, объяснение, поиск ошибок, проверка.

Цепочка занятия: **вопрос** → **таблицы** → **ключи** → **запрос** → **результат** → **проверка**

Ведомость в одной таблице

Так выглядят результаты, если хранить всё в одном листе: каждая строка повторяет студента, дисциплину и преподавателя.

Студент	Группа	Дисциплина	Преподаватель	Оценка
Кира Фадеева	М3401	Базы данных	Наталья Осипова	5
Лев Гурьев	М3401	Базы данных	Наталья Осипова	4
Виктория Смирнова	М3202	Базы данных	Наталья Осипова	5
Глеб Новиков	М3202	Базы данных	Наталья Осипова	3
Алина Воронова	М3201	Базы данных	Наталья Осипова	5

… и ещё 9 строк

? Наталья Осипова — в 6 из 14 строк. Что будет, если исправить её фамилию только в одной?

Три аномалии одной таблицы

Обновление

- фамилию исправили в одной строке из шести;
- один человек выглядит как двое.

Вставка

- нового преподавателя некуда записать,
- пока у него нет ни одной оценки.

Удаление

- удалили единственную оценку —
- вместе с ней пропали сведения о потоке и преподавателе.

Анна Миронова ведёт лекцию по линейной алгебре, но **в ведомости её нет совсем**: оценки по этой дисциплине ставят на практиках.

Зачем база, если есть электронная таблица

Электронная таблица хороша для разового расчёта. Когда данные обновляются и описывают несколько видов объектов, их удобнее хранить в базе.

Критерий	Электронная таблица	Реляционная база
Небольшой разовый список	удобна	может быть избыточна
Несколько связанных сущностей	повторы и аномалии	каждая сущность — своя таблица
Совместная работа	возможны конфликты	доступом управляет СУБД
Регулярные вопросы	много ручных шагов	запрос SQL повторяется

SQLite и PostgreSQL — это **СУБД**. В тренажёрах работает SQLite прямо в браузере, в лабораторной — PostgreSQL через DBeaver.

Что означает одна строка

Таблица — набор однотипных строк. Перед запросом выясните, что описывает одна строка: от этого зависит смысл COUNT(*).

students	**одного студента**	10 строк
streams	**один поток: лекцию или практику**	8 строк
enrollments	**одну запись студента на поток**	34 строки
results	**один результат аттестации**	14 строк

? В enrollments 34 строки. Значит ли это, что на потоки записаны 34 студента?

Первичный и внешний ключ

PRIMARY KEY

- однозначно определяет строку;
- не повторяется;
- не бывает NULL.

FOREIGN KEY

- ссылается на ключ другой таблицы;
- значение может повторяться;
- задаёт связь между строками.

```
students.student_id      →   PK   один студент
results.result_id        →   PK   один результат
results.student_id       →   FK   ссылка на students.student_id
enrollments(student_id, stream_id)  →  составной PK
```

? Почему ФИО студента плохо подходит на роль первичного ключа?

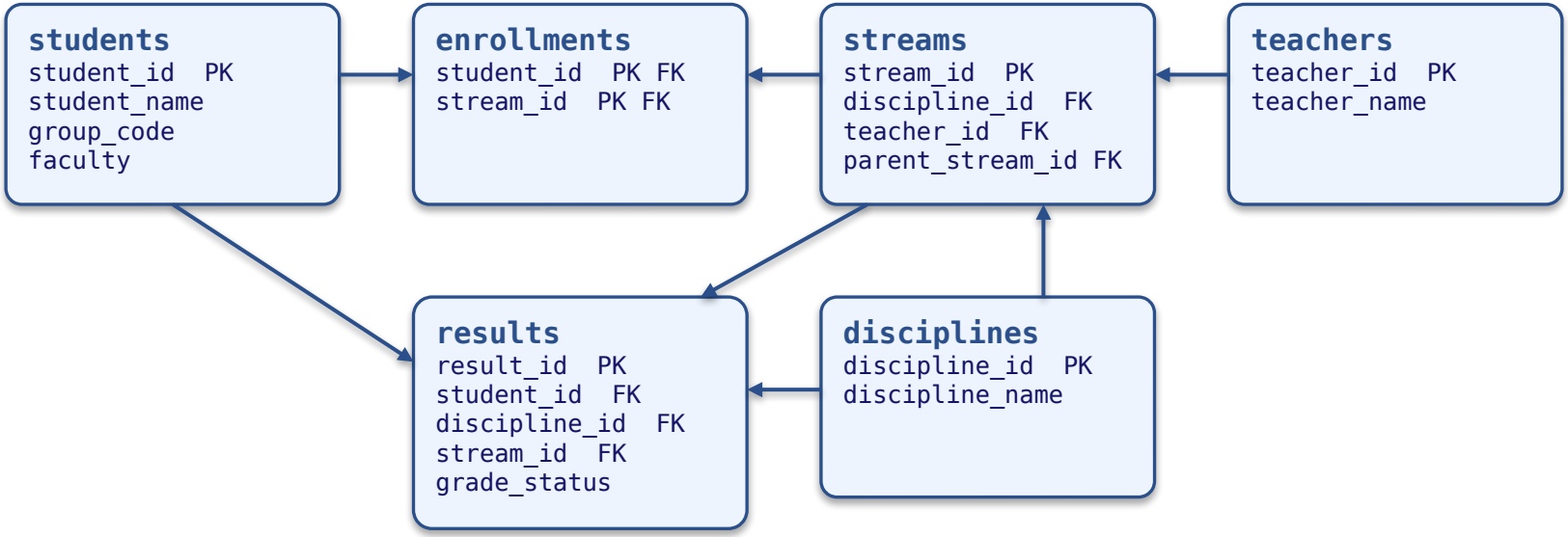
Связи между таблицами

Кардинальность показывает, сколько строк одной таблицы соответствует строке другой.

Один к одному	студент и его единственная зачётная книжка
Один ко многим	один студент — много результатов; одна дисциплина — много потоков
Многие ко многим	студенты и потоки: студент ходит на несколько потоков, в потоке много студентов

Связь «многие ко многим» хранится в промежуточной таблице **enrollments**: каждая строка — пара «студент — поток».

Схема учебной базы: результаты дисциплин



стрелка ведёт от «одного» к «многим»; parent_stream_id: практика → своя лекция

Основная база в DBeaver (схема **grades**) устроена так же, только данных в ней на порядки больше.

Первый SQL-запрос: SELECT и FROM

SQL — декларативный язык: вы описываете нужный результат, а СУБД решает, как его получить.

```
SELECT student_name, group_code, faculty
FROM   students;
```

SELECT — какие столбцы вернуть

FROM — из какой таблицы брать строки

***** — все столбцы: удобно для знакомства, но не для отчёта

Результат запроса — **тоже таблица**: её можно отсортировать, посчитать и проверить.

Фильтрация: WHERE

```
SELECT student_name, group_code
FROM   students
WHERE  faculty = 'ФЕН';            -- 4 строки
```

`=  <  >  <=  >=  <>`
сравнение

`AND  OR  NOT`
логические условия

`IN (…)`
одно из нескольких значений

`BETWEEN … AND …`
диапазон включительно

`LIKE`
поиск по шаблону

`IS NULL / IS NOT NULL`
проверка пропуска

? Почему пропуски ищут через IS NULL, а не через = NULL?

Сортировка и ограничение: ORDER BY, LIMIT

```
SELECT   result_id, student_id, score_value
FROM     results
WHERE    score_value IS NOT NULL
ORDER BY score_value DESC
LIMIT    5;
```

- **ORDER BY** задаёт порядок строк: ASC — по возрастанию (по умолчанию), DESC — по убыванию.
- **LIMIT** срабатывает после сортировки и оставляет первые строки.
- **У строк нет естественного порядка** — его задаёт только ORDER BY.

В PostgreSQL при DESC **пустые значения встают первыми** — поэтому в запросе есть WHERE score_value IS NOT NULL.

NULL — не ноль и не пустая строка

Ошибка

```
SELECT result_id, grade_status
FROM   results
WHERE  grade_value = NULL;
```

→ 0 строк

Верно

```
SELECT result_id, grade_status
FROM   results
WHERE  grade_value IS NULL;
```

→ 2 строки: absent и unknown

- NULL означает «значение неизвестно»;
- любое сравнение с NULL даёт «неизвестно», а не «истину»;
- агрегаты AVG, SUM, MIN, MAX пропускают NULL.

Неявка — не оценка «0». Если заменить NULL нулём, средняя по матстатистике **упадёт с 5,0 до 2,5**.

Агрегатные функции

Агрегат сворачивает несколько строк в один показатель.

COUNT(*)	количество строк	**AVG(поле)**	среднее
COUNT(поле)	количество непустых значений	**MIN(поле)**	минимум
SUM(поле)	сумма	**MAX(поле)**	максимум

```
SELECT COUNT(*), COUNT(score_value), AVG(score_value)
FROM   results
WHERE  discipline_id = 3;        -- «Базы данных»: 6 · 5 · 84,8
```

COUNT(*) считает строки, COUNT(поле) — непустые значения. По «Базам данных» 6 результатов, а баллов только 5.

GROUP BY и HAVING

```
SELECT   discipline_id, COUNT(*) AS graded_count
FROM     results
WHERE    grade_status = 'graded'   -- фильтр строк до группировки
GROUP BY discipline_id
HAVING   COUNT(*) >= 4             -- фильтр групп после расчёта
ORDER BY graded_count DESC;
```

WHERE

- фильтрует строки до группировки

HAVING

- фильтрует группы после агрегатов

? Можно ли записать COUNT(*) >= 4 в WHERE?

Логический порядок выполнения запроса

1	FROM / JOIN	results: 14 строк
2	WHERE	grade_status = 'graded': 12 строк
3	GROUP BY	по discipline_id: 4 группы
4	HAVING	COUNT(*) >= 4: 2 группы
5	SELECT	discipline_id и graded_count
6	ORDER BY	сначала дисциплина 3 (5), потом 1 (4)
7	LIMIT	в этом запросе не задан

Запрос пишется в одном порядке, а выполняется в другом. **Агрегат нельзя фильтровать в WHERE** — на этом шаге групп ещё нет.

Объединение таблиц: JOIN

JOIN нужен, когда поля ответа лежат в разных таблицах. ON задаёт, по каким ключам сопоставлять строки.

```
SELECT s.student_name, d.discipline_name, r.grade_value
FROM   results AS r
JOIN   students    AS s ON r.student_id    = s.student_id
JOIN   disciplines AS d ON r.discipline_id = d.discipline_id
WHERE  r.grade_status = 'graded'
ORDER BY d.discipline_name, s.student_name;
```

12 результатов graded → **12 строк**: каждый результат нашёл ровно одного студента и одну дисциплину.

INNER JOIN и LEFT JOIN: нагрузка преподавателей

INNER JOIN

- остаются только совпавшие строки;
- строка без пары не попадёт в результат.

LEFT JOIN

- остаются все строки левой таблицы;
- справа без пары появляются NULL.

```
SELECT t.teacher_name, COUNT(r.result_id) AS results_count
FROM   teachers AS t
LEFT JOIN streams AS s ON s.teacher_id = t.teacher_id
LEFT JOIN results AS r ON r.stream_id  = s.stream_id
GROUP BY t.teacher_id, t.teacher_name;
```

INNER JOIN вернёт **4 преподавателей**, LEFT JOIN — 5: Анна Миронова с нулём. С COUNT(*) вместо COUNT(r.result_id) у неё оказалась бы 1.

Ловушка: соединение размножает строки

Вопрос: сколько студентов записано на каждую дисциплину? Каждый студент записан и на лекцию, и на практику.

Дисциплина	COUNT(*)	COUNT(DISTINCT e.student_id)
Линейная алгебра	8	4
Теория вероятностей	10	6
Базы данных	12	6
Математическая статистика	4	4

В базе всего **10 студентов** — на дисциплину их не может быть 12. Проверка здравым смыслом ловит ошибку раньше формул.

SQL вместе с ИИ: что можно поручить

Поручить ИИ

- черновик запроса по тексту задания;
- объяснение чужого запроса по частям;
- причину ошибки по сообщению СУБД;
- проверочные запросы.

Где ИИ ошибается

- путает единицу наблюдения и размножает строки;
- заменяет NULL нулём «для удобства»;
- придумывает столбцы, которых нет в схеме;
- пишет на другом диалекте SQL.

Запрос, который выполнился без ошибок, **ещё не правильный ответ**.

Как поставить задачу ИИ

Пример запроса к модели

```
СУБД: PostgreSQL, схема grades.
Схема таблиц: <вставить CREATE TABLE>.
Значения grade_status: 'graded', 'absent', 'unknown'.
Задание: сколько студентов записано на каждую дисциплину.
Нужны столбцы: discipline_name, students_count.
Используй только эти таблицы и поля. Объясни каждую часть запроса.
```

- диалект — чтобы не получить синтаксис другой СУБД;
- схема и значения статусов — чтобы модель не придумывала поля;
- ожидаемые столбцы — чтобы результат можно было сверить.

В тренажёре 08·2 кнопка **«Задание для ИИ»** копирует схему и условие — останется вставить их в чат.

Проверка результата и типичные ошибки

Что проверить после запроса

- число строк до и после JOIN;
- контрольную сумму: 2 + 2 + 2 + 1 + 1 = 8 потоков;
- несколько строк вручную;
- объекты без связей через LEFT JOIN;
- здравый смысл: студентов не больше 10.

Частые ошибки

слишком много строк
→ JOIN размножил строки

пропали объекты без связей
→ нужен LEFT JOIN

агрегат в WHERE
→ его место в HAVING

= NULL
→ IS NULL

поле есть в двух таблицах
→ добавить псевдоним

Синтаксически верный запрос может давать **неверный по смыслу результат** — свой или предложенный ИИ.

Практика и лабораторная

Тренажёр 08·1 · без кода

- 4 предметные области;
- ключи, связи, данные и 5 запросов;
- «что если…» и объяснение ИИ.

Тренажёр 08·2 · код

- 19 заданий на учебной базе;
- поиск ошибок и проверка запросов ИИ;
- ответ проверяется по результату.

Лабораторная · DBeaver

- основная база edu, схема grades;
- индивидуальный вариант;
- PDF-отчёт и декларация ИИ.

Материалы темы: **rustam-azimov.github.io/data-ai-sem2-tema08**

Что мы разобрали

- почему одной таблицы мало и какие аномалии она создаёт;
- первичные и внешние ключи, связи и промежуточные таблицы;
- выборку: SELECT, WHERE, ORDER BY, LIMIT и работу с NULL;
- агрегаты, GROUP BY, разницу между WHERE и HAVING;
- INNER JOIN и LEFT JOIN, размножение строк;
- как ставить задачу ИИ и проверять его запросы.

SQL начинается с понимания схемы и заканчивается **проверкой результата**.